

ECE 792 Final Report

Project Title: Toward a tool to Evaluate Visual Behavior in Multi - device Environments

Team Member: Xihong Su

ECE Faculty Advisor: Dr. Andrew L Kun

Courses Involved: MATH 426, CS 515, MATH 527, CS620
ECE 602, ECE 647, ECE 724, ECE 757

Date: Monday, May 7, 2018

Project Completion: May 2018

Executive Summary

This thesis provides a method to understand visual behavior of two users engaged in a collaborative task around a table. We use visual behaviors of two users to identify activities that they are engaged in. This method presents where two users cast their gaze, when two users cast their gaze, how long they look at a particular target, as well as how these behaviors are related between users.

The hardware that the experiments require are eye tracker and a laptop. The software that the experiments require are pupil capture, pupil player and Matlab. Pupil capture is used to do recordings, and pupil player is used to export recordings and visualize data, and Matlab is used to do all calculations and drawing figures.

Main idea of this thesis is from the paper of Richardson and Dale [1]. One participant is invited to pretend two users engaged in a collaborative task around an experiment table. Results show how visual behaviors are related between two users: 1) the gazes of the two users can follow each other perfectly around the table over time; 2) the gaze of the two users don't follow each other perfectly around the table over time; 3) the gaze measurement data includes noisy measurements which makes it difficult to determine how the visual behaviors of the two users are related to each other.

Contents

1. Project Overview	4
1.1 Problem Definition.....	4
1.2 Design Architecture	4
2. Work Completed	5
2.1 Surface Registration	5
2.2 Calibration	7
2.3 Data Recording	9
2.3.1 Eye Tracking Headset.....	9
2.3.2 Pupil Capture	10
2.3.3 Data Collection.....	12
2.4 Export Data.....	12
2.5 Data Analysis	15
2.5.1 Frame Image	15
2.5.2 Pupil Position and Gaze Position.....	16
2.5.3 Gaze Position and Surfaces.....	17
3. Result Analysis.....	18
3.1 Matlab Programming	18
3.2 Result Analysis of Collaborative Task	19
3.3 Result Analysis of Imperfection Gaze follow	21
3.4 Result Analysis of Imperfection Measurements.....	21
4. Conclusion	22
5. Future Work.....	23
6. Acknowledgements	23
7. Appendices.....	24
7.1 Proposed Timelines.....	24
7.2 Actual Timelines Achieved	25
8. Reference	26

1. Project Overview

1.1 Problem Definition

Human visual behavior has significant potential influence for activity recognition. Our eye movements are closely linked to our goals, tasks, and intentions. Bulling explored a series of eye-based activity recognition [2]. Recent research developments on eye-tracking show that gaze can be used to reflect cognitive and collaborative processes at various time scales [3]. We are interested in understanding visual behavior of multiple users engaged in a collaborative task around a table. We hope to use visual behavior to identify activities that multiple users are engaged in. However, we don't know where multiple users cast their gaze, and when multiple users cast their gaze, and how long users cast their gaze, and how these behaviors related between users. We use one participant to pretend two users engaged in a collaborative task around a table over time. We find where these two users cast their gaze, and more importantly when multiple users cast their gaze, how long they look at a particular target, as well as how these behaviors are related between users.

1.2 Design Architecture

Main idea of this project is from the paper of Richardson and Dale [1]. The design of our system shown in figure 1 includes five parts: surface registration, calibration, data recording, export data and data analysis. Surface registration includes what size markers should be and how many markers should be used to identify a surface. The calibration process is to establish a mapping from pupil to gaze coordinates and know what a participant is looking at. We need to make sure that the participant's pupil is properly tracked [4]. Pupil capture is used to detect and track markers in our experimental environment, record video and events. Surface tracking in pupil capture is used to track different surfaces during the experiments. Pupil player is used to visualize

data and export recording data. Image processing is used to give frame image information of the exported video. Matlab is chosen to process gaze positions, surface information, and draw figures.

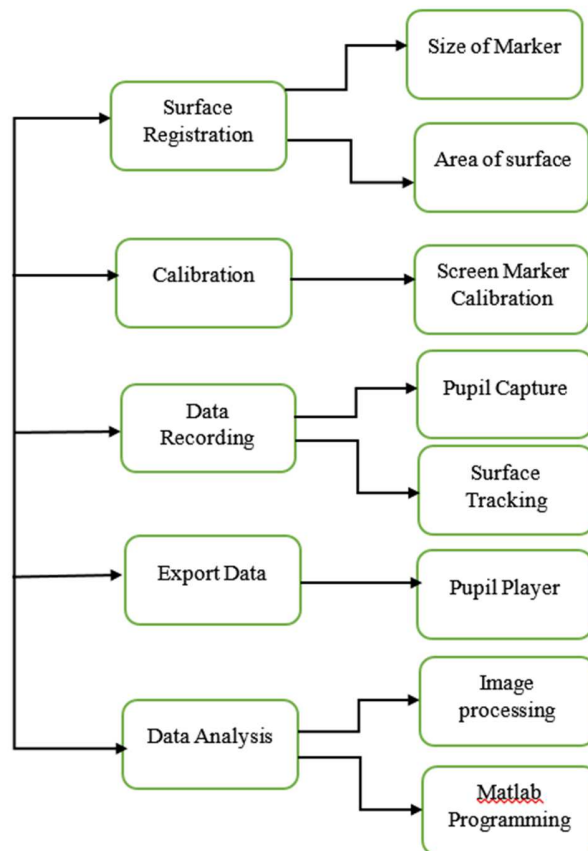


Figure 1 General View of the System

2. Work Completed

2.1 Surface Registration

Surfaces are targets that a participant is looking at during the experiments. We want to know where the participant cast his gaze. First, we need to know the area of each surface. Second, we can know which surface the participant is looking at. Pupil labs provide a marker tracking plugin to allow us to register surfaces in pupil capture and track each surface in real time during the recordings. Pupil labs provides 64 unique markers, as shown in figure 2. The markers can be scaled to different sizes for satisfying different experiment requirements. When printing markers on paper,

we need to have white border width of at least 1.2 x the marker grid size for marker stickers [5].

For this project, we use 3cm x 3cm square printed markers for the experiments.

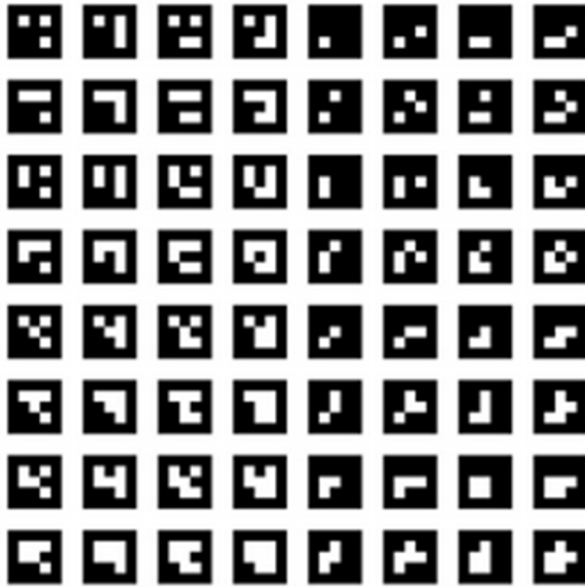


Figure 2 Markers

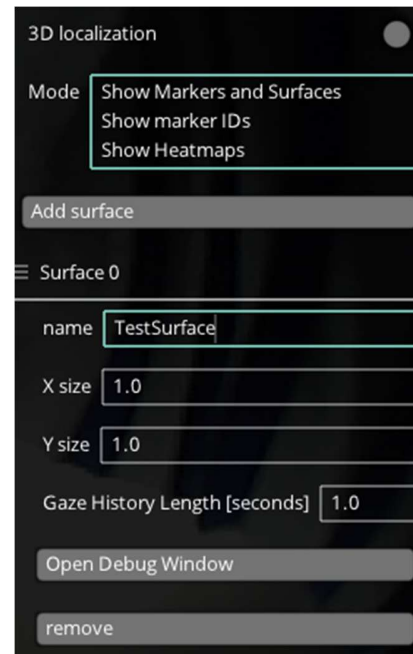


Figure 3 Adding a Surface

Surfaces can be registered with pupil capture in real time. Surfaces can also be registered offline with pupil player. We register a surface in pupil capture in real time shown in figure 3.

There are three modes: show markers and surfaces, show marker IDs and show heatmaps. For the mode of heatmaps, the name of surface 0 is “TestSurface”. The X size and Y size are used to define a grid of a surface in “show heatmaps” mode. The “Gaze History Length” describes how many recent gaze positions can be used to calculate the heatmap [6]. Heat maps is mainly used for the agglomerated analysis of the visual exploration patterns in a group of users [7].

This project focuses on “show markers and surfaces” and “show marker IDs”. The mode of “show Marker IDs” is shown in figure 4. “Draw markers” draws a rectangle around each marker and an arrow to show orientation. Each marker detected displays its ID, orientation, and

center [5]. In theory, all of 64 unique markers should be detected by pupil capture and eye tracker. However, 62 out of 64 have been detected during the experiments.

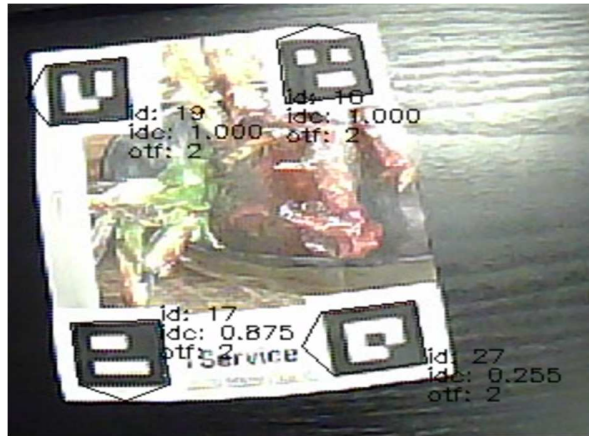


Figure 4 Marker IDs



Figure 5 A surface

The mode of “show markers and surfaces” is shown in figure 5. A surface can be defined by one or more markers. For this project, we use four printed Id markers to define one surface, and the surface detected is displayed with a red line and a triangle that signifies the orientation of the surface. “4/4” means that four markers are used to define the surface and all four markers are detected. If we cover one marker, three out of four (3/4) markers will be detected, but the same surface can still be detected. All surfaces defined in pupil capture are automatically saved, these registered surfaces will appear when we start the marker tracking plugin [5]. Each surface is 11cm x 13cm, and six surfaces are registered for the experiments.

2.2 Calibration

The calibration process is to establish a mapping from pupil to gaze coordinates and know what a participant is looking at. Calibration is a necessary step to get data of eye movements with high confidence. Before calibrating, our eyes are well positioned for a robust eye tracking performance. Open eye windows “Detect eye 0” and “Detect eye 1” in the plugin “General”. If the pupil is detected, we can see a red circle around the edge of our pupils and a red

dot at the center of our pupils. When the algorithm's detection confidence is high, the red circle will become opaque, otherwise the circle will become more transparent. Several possible eye positions in pupil capture are shown in figure 6.

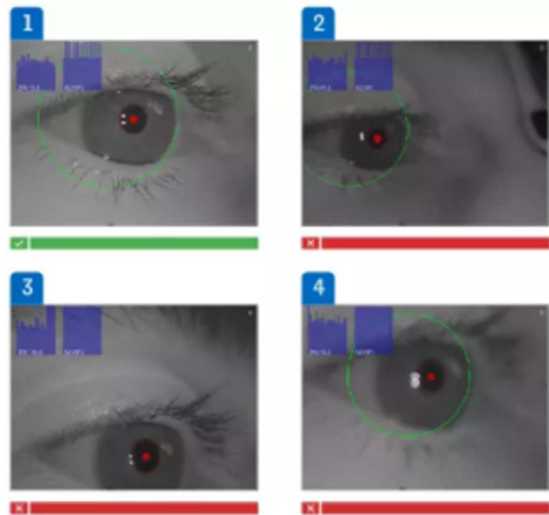


Figure 6 Eyes Positions

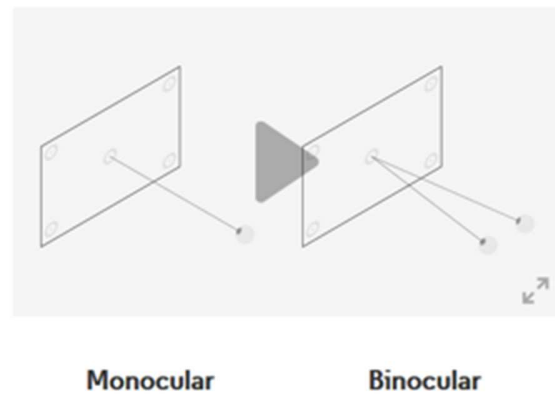


Figure 7 Calibration with eye camera

For figure 6, the eye in subfigure 1 is in focus and all range of the eye movements are visible. In subfigure 2, the camera arm is too far away from the eye. In subfigure 3, the eye is not centered in the frame. In subfigure 4, the eye is out of focus in subfigure 4. We adjust headset to get good eye video [8]. After the user's pupil is properly tracked, we make sure that the world camera is in focus for the distance at which the participant wants to calibrate. The participant can see the entire area that he wants to calibrate within the world cameras extents(FOV) [6].

“Calibration can be conducted with a monocular or binocular eye camera setup.”, as show in figure 7. For this project, we use calibration with binocular eye camera setup [8].

Calibration methods include screen marker calibration and manual marker calibration. Screen maker calibration is the default method, and it is best suited for close range eye-tracking in a narrow field of view. Manual marker calibration is done with an operator and a subject. “It is

suited for midrange distances and can accommodate a wide field of view. The operator will use a printed calibration marker.” [8] For this project, the participant sitting around the table looks at six surfaces on the table in some orders. Therefore, screen marker calibration is suited for this project.

The detection & mapping mode can work in 2d, 3d, and disable. If it works in the 3d mode, we can achieve 1.5-2.5 deg of accuracy of calibration. We set detection & mapping to work in 3d mode for the screen marker calibration. We did screen maker calibration on the 13.3 inches Dell laptop. Select screen maker calibration and press c on the keyboard and follow the marker on the screen with the participant’s eyes. Keep the participant’s head still during calibration. The calibration is only accurate inside the field of view we have calibrated. For example, if the user only looked at the marker that are in the left half during the calibration, we will not get good accuracy in the right half. Because the calibration is done on the 13.3 inches Dell laptop, six surfaces can fit in the field of view we have calibration. Then we can get good accuracy with these six surfaces [8].

2.3 Data Recording

2.3.1 Eye Tracking Headset

Pupil mobile eye tracking headset is shown in figure 8, and we also call it eye tracker. The author wears the eye tracker and looks at six registered surfaces on the table in the lab. Actually, the data used in the project is collected from the participant. The eye tracker is like a pair of glasses. It has three cameras: one world camera and two eye cameras. World camera records the subject’s field, and an eye camera records a subject’s eye movements. In order to know where the participant is looking at, calibration is used to correlate these two streams of information. We can rotate the world camera up and down and slide the eye camera arm along the track to get optimal eye

detection and world camera FOV [9]. The eye tracker is connected to the Dell laptop via a USB 2.0 connector.



Figure 8 Eye tracker

2.3.2 Pupil Capture

“Pupil Capture is the software used with the Pupil Headset.” The software is used to read the video streams coming in from the world camera and the eye camera. “Pupil capture uses the video streams to detect your pupil, track your gaze, detect and track markers in your environment, record video and events, and stream data in real time.” [8] Pupil capture can work in Linux operating system, macOS, and Windows. The operating system of my Dell laptop is Windows 8.1 Pro, so we use `pupil_v1.5-12_windows_x64.7z` that was released on 03/02/2018 for the experiments. The capture window in figure 9 is the main control center for pupil capture.

In figure 9, area 1 contains clickable buttons for plugins. C button is for calibration, T button is for gaze prediction accuracy test, R button is for recording, S button is surface tracking, and A button is for adding surfaces. Area 2 by default contains the graphs CPU, FPS (frames per second), and pupil algorithm detection confidence. We set FPS to 30 because we use USB 2.0

connector that restricts the FPS. Useful data should carry a confidence value greater than 0.7. Area 3 is the menu, and it contains settings and contextual information for each plugin.

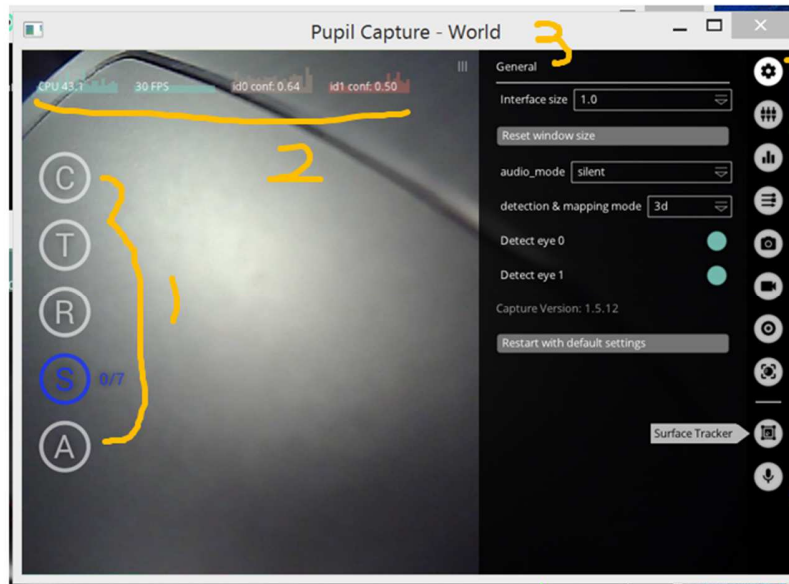


Figure 9 Pupil Capture

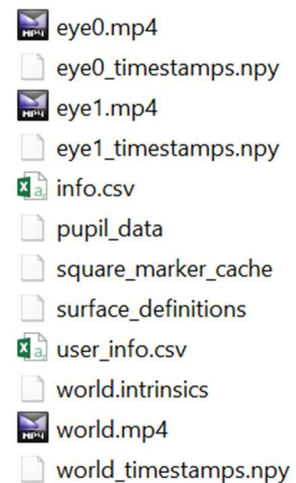


Figure 10 Format of one video

Area 4 contains clickable buttons for each plugin [8]. For the experiments, `audio_mode` is set to `silent`, and resolution is set to (640,480), and frame rate is set to 30. As to plugin recorder, the world video stream has two settings: more CPU smaller file and less CPU bigger file. When using the setting more CPU smaller file, the video is compressed using `ffmpeg`'s default settings. It can give a good balance between image quality and files size. "The frame rate of this file is set to your capture frame rate." When using the setting less CPU and bigger file, a video is compressed by the camera itself. "The 'frame rate' setting in the pupil capture controls the frame rate of the videos." [8] During experiments, we use the setting of more CPU smaller file. For plugin manager, we load plugin surface tracker because surfaces are identified during the recordings. Other settings in pupil capture are set by default.

2.3.3 Data Collection

We have registered six surfaces, and each surface is marked by four markers. We invite one participant to pretend two users engaging in a collaborative task around a table. The participant did screen marker calibration and looked at six surfaces in three orders. The first order is surface 1,2,3,4,5,6, and the second order is surface 1,2,3,6,5,4, and the third order is surface 6,5,4,3,2,1. We recorded many videos that the participant looked at surfaces in the first order and chose two of them to pretend two users engaged in a collaborative task.

We recorded some videos that the participant looked at surfaces in the second order and in the third order. We choose one video from the first order and another video from other orders, and then compare visual behaviors of these chosen two videos. This simulates two users finish the task separately.

Each video is between 40 seconds and 55 seconds. After the recording, the format of one video is shown in figure 10. There are two eye videos and one world view video. 0 represents left eye, and 1 represents right eye. It also includes square makers and surfaces that registered in pupil capture. World.mp4 is the video stream of the world camera view. world_timestamps.npy is 1 d array of timestamps for each world meta data.

2.4 Export Data

After recordings, we need to export recordings to analyze visual behaviors of the participant. Pupil player is used to look at recordings and visualize recording data and export it. Figure 11 is the main control center of pupil player, and it displays the video recorded by pupil capture on 03/11/2018. At the bottom of figure 11 is timeline events and timeline. Timeline events include pupil diameter, pupil confidence, marker cache. Timeline is used to control the playback of the video with the play/pause button.

In order to get useful information such as surfaces and user gaze, we load some plugins that can give more detailed information about the recordings before exporting data. For plugin “offline surface tracker”, we can see surfaces registered in pupil capture and change surface information such as name. For plugin “video export launcher”, the default name of the exported video is “world_viz.mp4”, and we can change its name. For plugin “raw data exporter”, we can select an export frame range using the trim marks in the seek bar. For plugin “gaze data from recording”, option “manual correction” allows us to apply a fixed offset to my gaze data. By default, “x_offset” and “y_offset” are 0.00. Another option is gaze producer, and the default setting is “gaze from recording.” We use these default settings for experiments. Plugins with the vis prefix are called visualization plugins.

Visualization plugins include vis circle, vis cross, vis fixation, vis light points, vis polyline, and vis watermark. They can be additive and directly operate on the gaze positions to produce visualizations. We choose to visualize the gaze positions with a circle plugin that has the following parameters: radius, stroke width, fill and color. Radius is the radius of the circle around the gaze point, and stroke width defines the thickness of the stroke in pixels, and there are red, green, blues values for color [8]. The green circle in figure 11 is the gaze position of the participant. Several red square lines describe how the system identifies those surfaces. For ID markers, four markers have been detected in surface 1, and two markers have been detected in surface 2. Even though only two markers have been detected on surface 2, area of the surface detected is the same as the one with four markers.

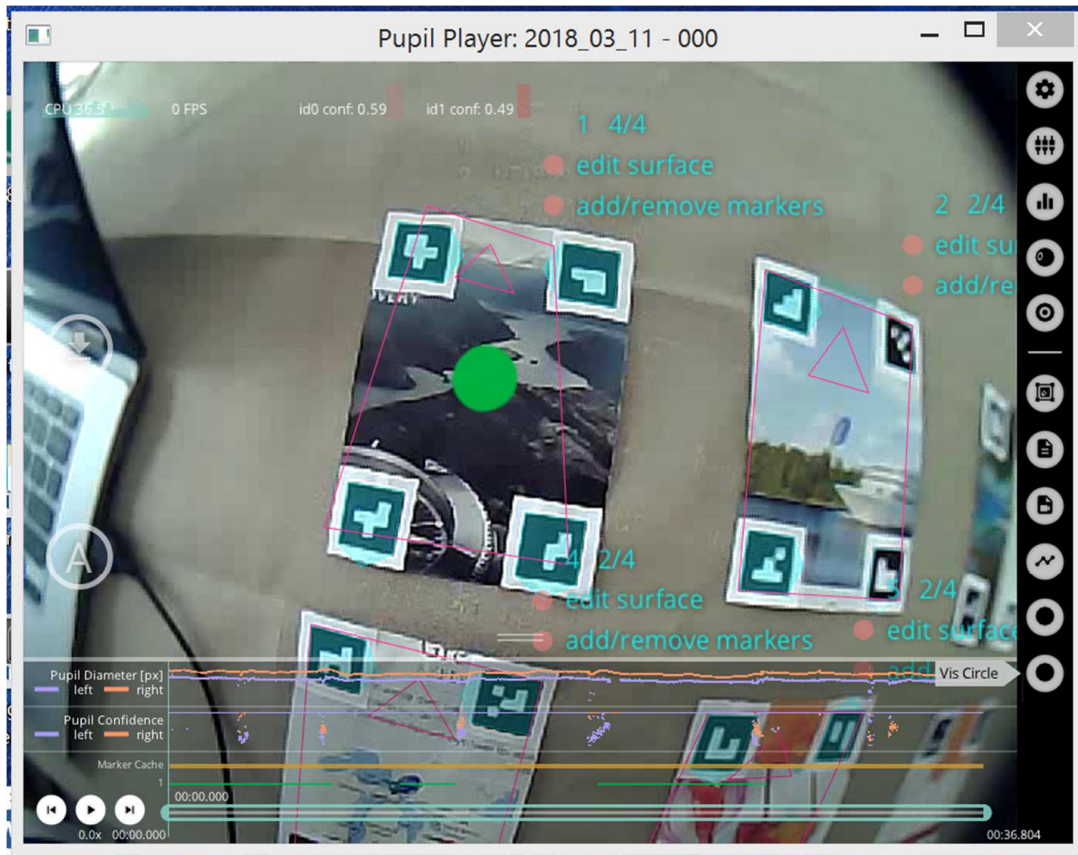


Figure 11 Pupil player

Press the e button on the keyboard to start the export. All exports will be saved within the exports sub-directory within the recording directory [8]. The exported file is shown in figure 12. We can play with the world video world_viz.mp4 to see the visual behavior of the participant. We have registered six surfaces in pupil capture, so the pupil player can generate surface visibility reports. The surface fold includes fixations_positon_on_surface_ surfaceName.csv, gaze_positions_on_surface_ surfaceName.csv, heatmap_ surfaceName.png, srf_positions_ surfaceName.csv, surface_events.csv, surface_gaze_distribution.csv and surface_visibility.csv. “surfaceName” is the name of a surface.



Figure 12 The exported file

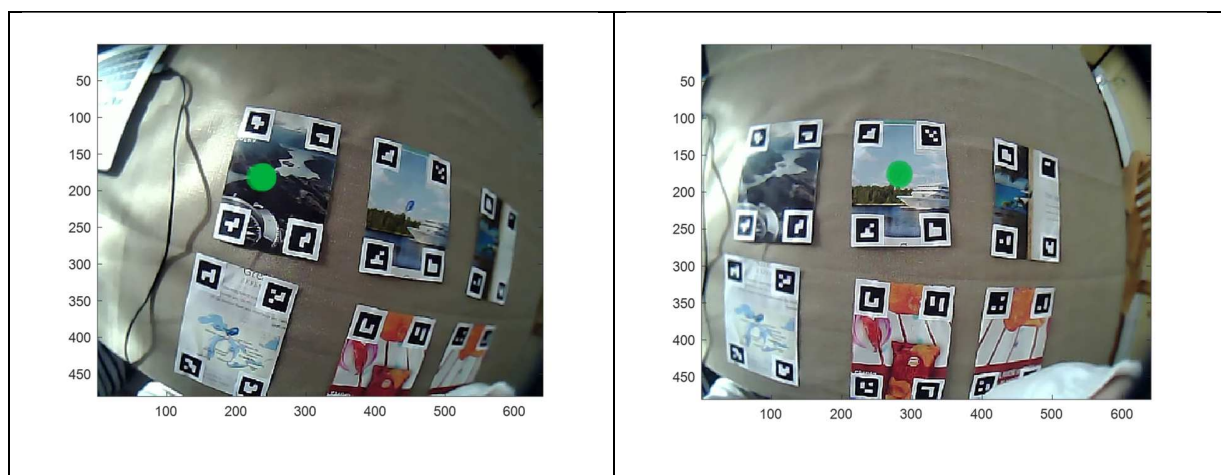
2.5 Data Analysis

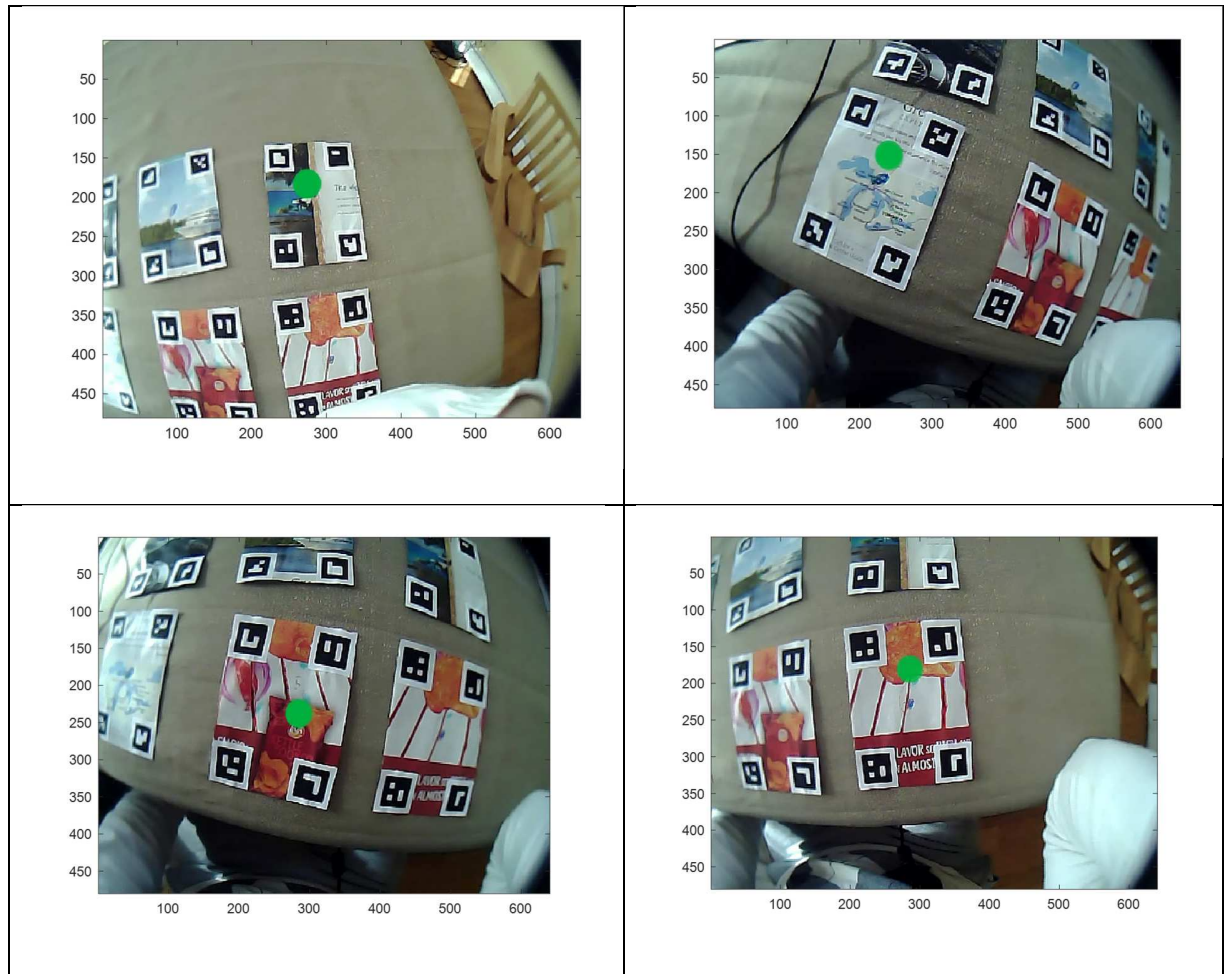
We analyze the exported video to get frame images and find where the participant cast his gaze at specific timestamp. We also analyze useful data items of exported files to get eye movements of the participant over time.

2.5.1 Frame Image

We can play with the video world_viz.mp4 to see the visual behavior of the participant. Table 1 shows gaze points of the participant on six surfaces on the table. Each subfigure in table 1 is one frame of the exported video. The unit of the frame is one pixel. The origin is “top left” and “bottom right” is the maximum x, y. The participant sits around the table, and the green circle is the participant gaze.

Table 1 Frame images and gaze positions on six surfaces





2.5.2 Pupil Position and Gaze Position

Pupil position is the coordinates of the pupil center in the eye video. “The pupil position gets mapped into the world space and thus becomes the gaze position. This is the current center of the subject visual attention-or what you’re looking at in the world.” There are data items “timestamp”, “index”, “id”, “confidence”, “norm_pos_x”, “norm_pos_y”, “base_data”, “gaze_point_3d_x”, “gaze_point_3d_y” and “gaze_point_3d_z” in gaze_positions.csv. Confidence is used to assess how accurate on this measurement. 1 represents perfect confidence. “gaze_point_3d_x” is the x position of the 3d gaze point (the point the subject looks at) in the world camera coordinate system, “gaze_point_3d_y” is the y position of the 3d gaze point, and “gaze_point_3d_z” is the z position of the 3d gaze point. “norm_pos_x” is the x position of gaze

in the world image frame in normalized coordinates, and “norm_pos_y” is the y position of gaze in the world image frame in normalized coordinated [8]. A normalized coordinate system has the origin (0,0) at the bottom left and (1,1) at the top right [6].

2.5.3 Gaze Position and Surfaces

For “gaze_positions_on_surface_surfaceName.csv” file, surfaceName can be 1,2,3,4,5 and 6 because we have registered six surfaces in pupil capture. There are nine data items shown in figure 13. The unit of “world_timestamps” and “gaze_timestamp” is second(s). “x_scaled”

A	B	C	D	E	F	G	H	I
world_timestamp	world_frame_idx	gaze_timestamp	x_norm	y_norm	x_scaled	y_scaled	on_srf	confidence
411763.4068	0	411763.3055	0.467837	0.270052	0.467837	0.270052	TRUE	1
411763.4068	0	411763.3097	0.477456	0.268999	0.477456	0.268999	TRUE	1

Figure 13 Data items in gaze_positions_on_surface_surfaceName.csv

and “y_scaled” can be used in heatmaps, and we ignore them for this project. “x_norm” and “y_norm” are the mapped gaze positions in surface coordinates [10]. “on_surf ”shows if a gaze position on a surface, and its value is ‘True’ or ‘False’. Confidence describes how accurate the measurement is.

For “surface_events.csv file” file, there are five data items shown in figure 14. “surface_name” is a surface name we have registered in pupil capture. “surface_uid” is a surface name that the pupil player interprets each surface. “even_type” describes when the participant begins to cast his gaze on one surface and when his gaze leaves one surface.

	A	B	C	D	E
1	frame_number	timestamp	surface_name	surface_uid	event_type
2	0	411763.4068	1	1520791896	enter
3	0	411763.4068	4	1520792187	enter
4	0	411763.4068	5	1520792200	enter
5	3	411763.6393	1	1520791896	exit

Figure 14 Data items in surface_events.csv

From figure 14, it seems that the participant casts his gaze on surface 1, surface 4 and surface 5 at the same time. It doesn't make sense. When we check "gaze_positions_on_surface_1_xxx.csv", the value of "on_srf" is True, and the confidence is 1 at timestamp 411763.4068. For "gaze_positions_on_surface_4_xxx.csv", the value of "on_srf" is False, and the confidence is 1 at timestamp 411763.4068. For "gaze_positions_on_surface_5_xxx.csv", the value of "on_srf" is also False, and the confidence is also 1 at timestamp 411763.4068. Therefore, the participant actually casts his gaze only on surface 1 at timestamp 411763.4068.

3. Result Analysis

Main idea is from the paper of Richardson and Dale [1]. Richardson and Dale use cross-recurrence plot to analyze the relationship between speaker and listener. We collect data from one participant to simulate having two users and draw cross-recurrence plot to analyze how visual behaviors are related between two users. User one has one case, and user two has three cases, and so there are three comparisons between two users.

3.1 Matlab Programming

We choose Matlab to implement our algorithms because Matlab has several advantages over other languages. First, its basic data element is the matrix. Both Gaze data, surface data and image data can describe as matrixes. Table arrays can store column-oriented data in a variable. Table variables can have different sizes and data types. We can read data from a file into a table using the "readtable" function. Access properties using the form "tableName.properties.propertyName"[11]. Second, the graphical output is friendly and optimized for interaction. We can easily to draw 2d and 3d figures, and change colors, sizes, scales by using the graphical interactive tools. Third, the addition of toolboxes can greatly expand the functionality of Matlab. For example, image processing toolbox is useful for image analysis in this project [12].

3.2 Result Analysis of Collaborative Task

According to “gaze_positions_on_surface_surfaceName.csv”, we can get numbers of gaze on each surface, when the participant gaze cast on a surface, data confidence and so on. Data with confidence less than 0.7 is ignored. Time-surface plot and cross-recurrence plot are used to analyze how visual behaviors are related between two users.

The participant pretends two users engaging in a collaborative task around a table. Figures 15 and figure 16 show where the participant casts his gaze, when the participant casts his gaze and how long the participant casts his gaze. x axis is the time of the video, and y axis is the name of surfaces. There are six surfaces.

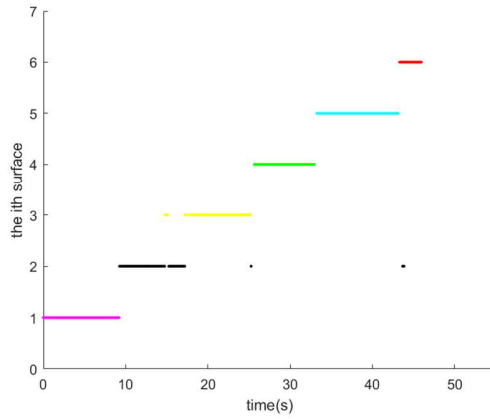


Figure 15 User one

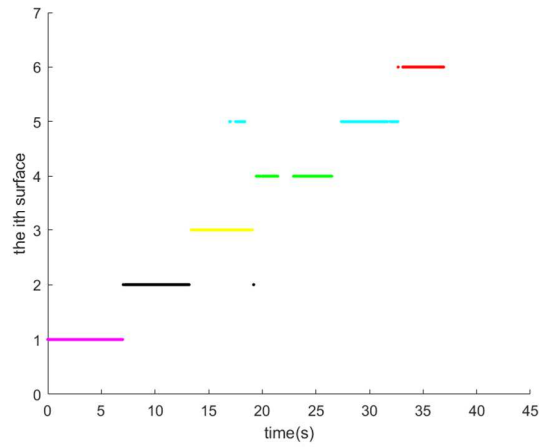


Figure 16 Pretended user two

Figure 15 describes that the user one casts his gaze on surface in the order 1, 2, 3, 4, 5 and 6. Average time of user one casting his gaze on each surface is about 7.7 seconds. Figure 16 describes that user two casts his gaze on surfaces in the order 1,2,3,4,5,6. The order that user two casted his gaze on surfaces is 1, 2, 3, 4, 5, and 6. The average time of the participant casting his gaze on each surface is about 6.5 seconds. User one and user two look at six registered surfaces in the same order, and the average time of user two is shorter than user one.

Cross-recurrence plot of these two users are shown in figure 17. X axis represents the time when user two looks at those six surfaces, and y axis represents the time when user one looks at those six surfaces, and time unit is second. According to figures 15 and 16, user one was looking at the first surface between 0 second and 8 seconds, and user two was looking at the first surface between 0 second and 7 seconds. In other words, user one and user two were looking at the first surface at almost the same time. This is why we have a short line that is [0: 7] on x axis and [0:8] on y axis in figure 17.

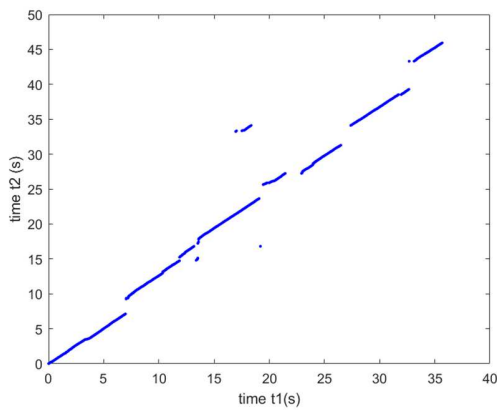


Figure 17 Two users work collaboratively

In theory, if two users cast their gazes on the same surface at the same time, the figure 17 would be a continuous straight line and the slope of the line would be 1. However, there are some discontinuities in figure 17. There could be two possible reasons. The first possible reason would be that two users don't cast their gaze on the same surface for some short time. Because we use one participant to pretend two users, and there is no strict time synchronization. The second possible reason would be that confidence of gaze points are lower than 0.7. When the confidence is lower than 0.7, the gaze point collected will be ignored.

Time t2 is longer than time t1, and the average of slopes of discontinuous lines is a little bigger than 1. However, if we connect the discontinuous points in figure 17, it can be approximated

as one line. Therefore, figure 17 shows that two users finish the task collaboratively and two users can follow each other perfectly around the table over time.

3.3 Result Analysis of Imperfection Gaze follow

For the second case of user two, the order that the second user casts his gaze is on surface 1, 2, 3, 6, 5, 4. The average time of the participant casting his gaze on each surface is about 9 seconds, shown in figure 18. According to figures 15 and 18, we know two users were looking at those six surfaces in different orders. Cross-recurrence plot of these two users are shown in figure

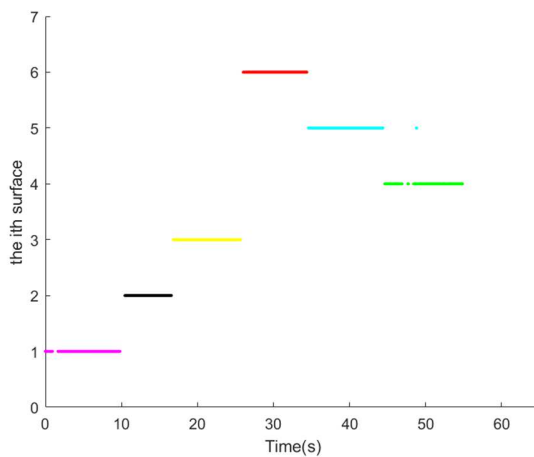


Figure 18 The second case of user two

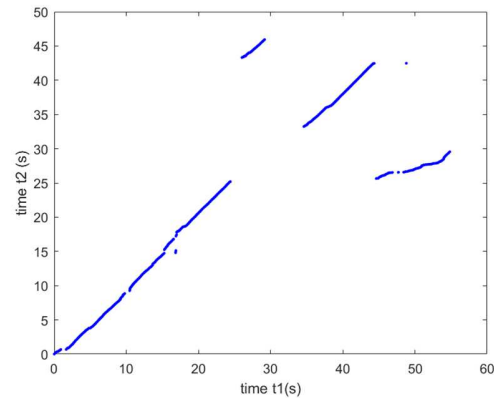


Figure 19 Two users work separately

19. There is an approximated one line between 0 second and around 25 seconds, and this shows that two users were looking at the first three surfaces at almost the same time. For the rest of time, there are three short lines, and this means two users were looking at different surfaces at the same time. Therefore, the gazes of the two users do not follow each other perfectly around the table over time.

3.4 Result Analysis of Imperfection Measurements

For the third case of user two, the order that the second user casts his gaze is on surface 6, 5, 4, 3, 2, 1. The average time of the participant casting his gaze on each surface is about 8.5

seconds, shown in figure 20. There is overlap between 25 seconds and 35 seconds. It seems that the participant was looking at surface 3 and surface at the same time. Actually, the line drawn by green circle is noise measurements. According to figures 15 and 20, we know two users were looking at those six surfaces in different orders. Cross-recurrence plot of these two users are shown in figure 21. There are six short parallel lines, and this shows that two users were looking at different surfaces at the around the same time. The gazes of the two users do not follow each other around the table over time. In addition, there is extra line drawn by green circle in figure 21. It shows that the gaze measurement data includes noisy measurements which caused by calibration and eye tracking device.

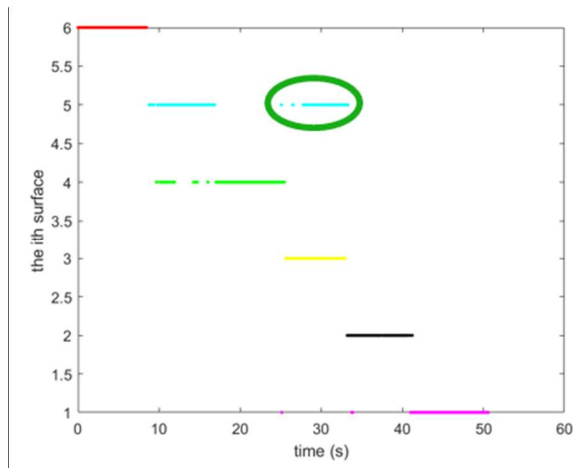


Figure 20 The third case of user two

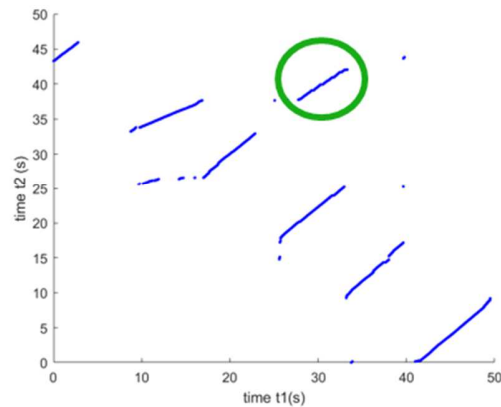


Figure 21 Noisy measurements

4. Conclusion

We seek to understand visual behavior of multiple users engaged in a collaborative task around an interactive surface. We collected data from one participant and used this data to simulate having two users engaged in a collaborative work. We designed software to visualize the gaze behavior of two users around a table. The visualization shows us how the timing of user gazes is related. We conducted controlled experiments to demonstrates visualizations for three cases: (a)

when the gazes of the two users follow each other perfectly around the table over time, (b) when the gazes of the two users do not follow each other perfectly around the table over time, and (c) when the gaze measurement data includes noisy measurements which makes it difficult to determine how the visual behaviors of the two participants are related to each other.

5. Future Work

The method we propose describes the visual behavior of using data to simulate two users over time. We can improve the results in two ways. First, we invite one participant to pretend two users engaged in a task collaboratively. When we do recordings, there is no strict time synchronization. One possible improvement would be to invite two participants and do time synchronization before recordings. Second, there is manual correction “x_offset” and “y_offset” in plugin “gaze data from recording.” We use the default value 0.00 for “x_offset” and “y_offset”. Gaze data can be influenced by some factors. It could be better if we can give non-zero values for “x_offset” and “y_offset”.

6. Acknowledgements

I wish to express my sincere thanks to Dr. Andrew Kun for providing me with all the necessary facilities for the research and for sharing valuable guidance and encouragement extended to me. I take this opportunity to express gratitude to all of the ECE Department faculty members for their help and support. I also thank my family for the unceasing encouragement, support and attention.

7. Appendices

7.1 Proposed Timelines

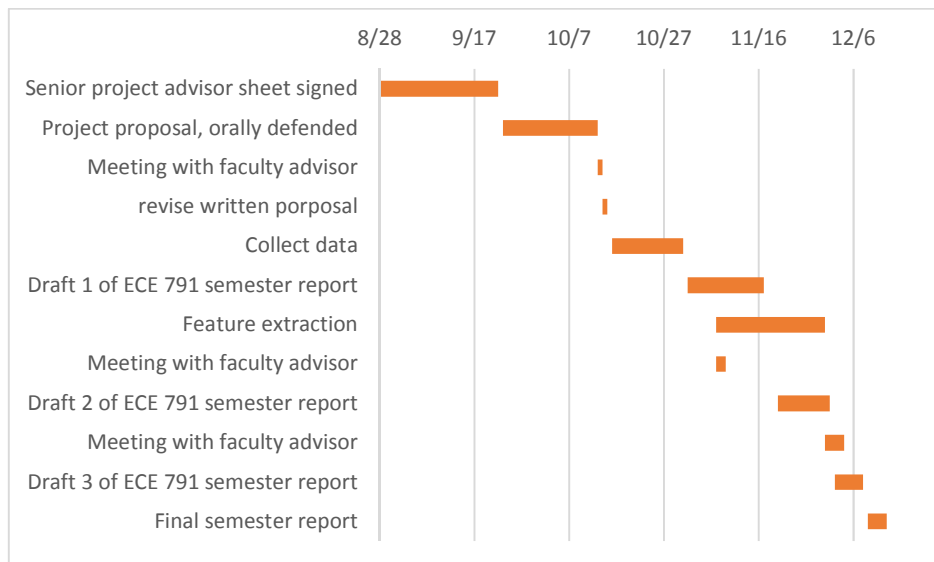


Figure 22 Proposed project timeline for fall semester

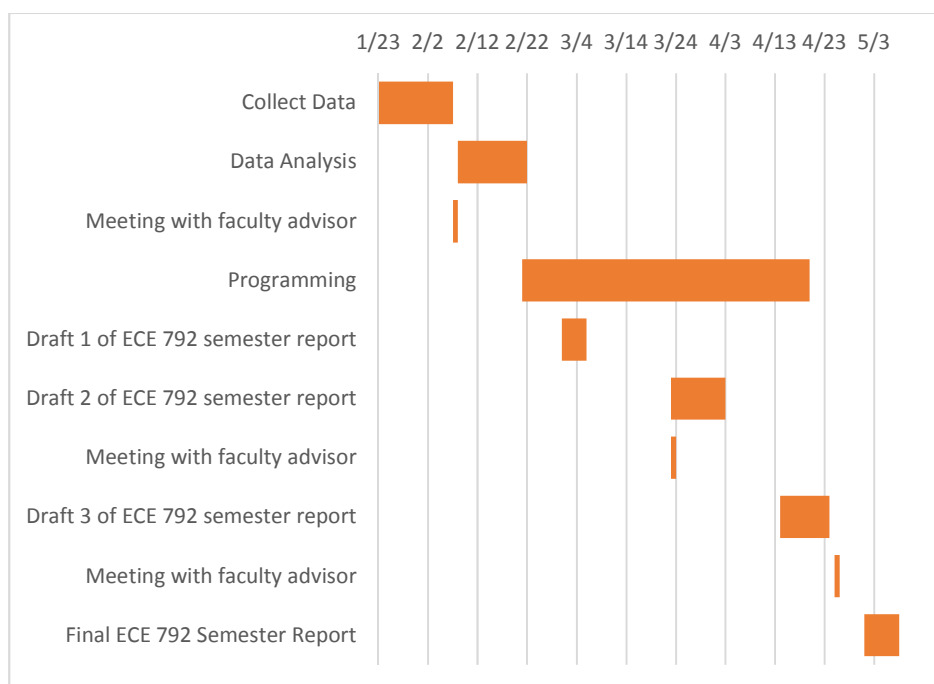


Figure 23 Proposed project timeline for spring semester

7.2 Actual Timelines Achieved

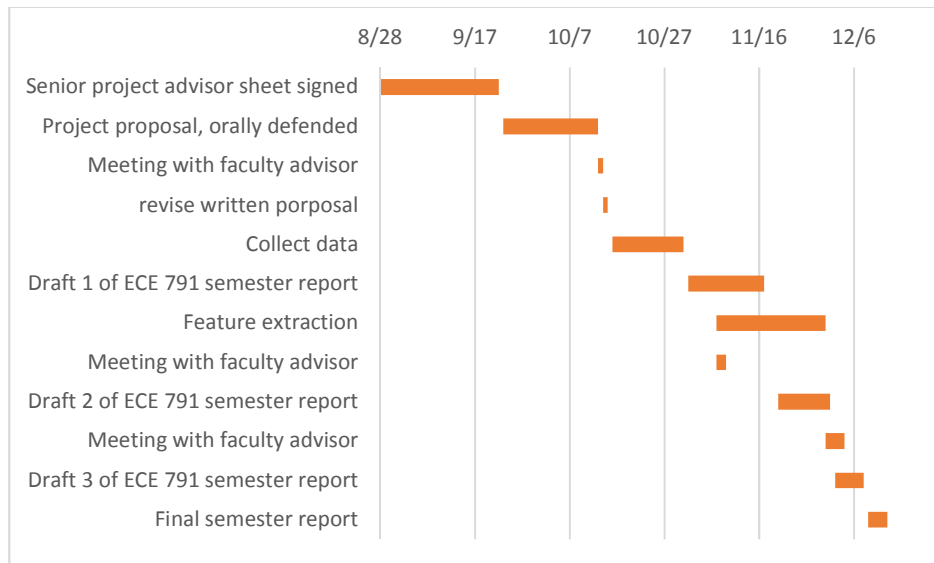


Figure 24 Actual project timeline for fall semester

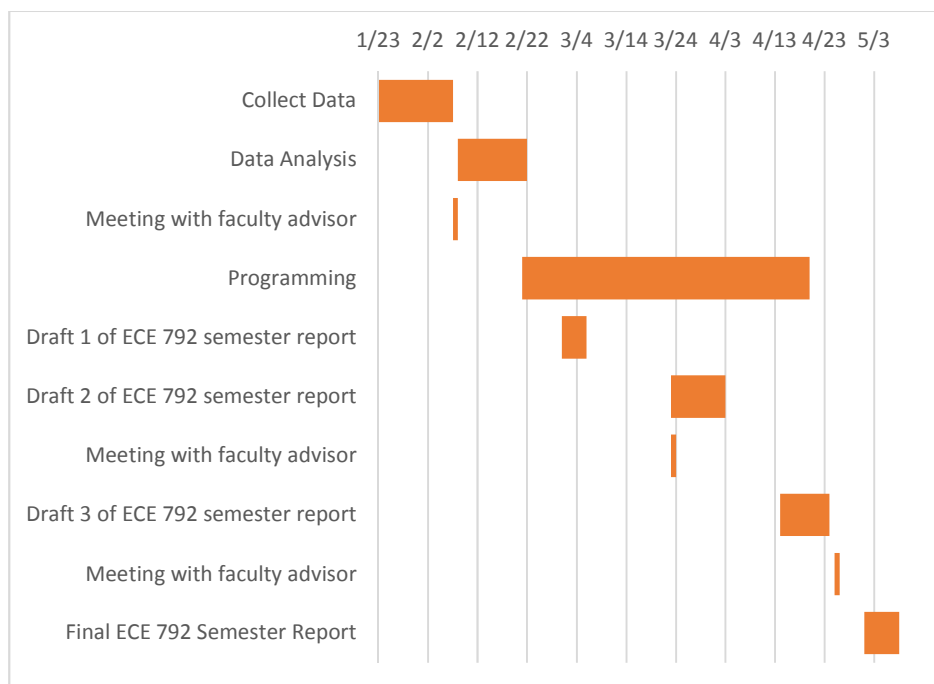


Figure 25 Actual project timeline for spring semester

8. Reference

- [1] Richardson, D. C., & Dale, R. (2005). "Looking to understand: The coupling between speakers' and listeners' eye movements and its relationship to discourse comprehension". *Cognitive science*, 29(6), pp.1045-1060.
- [2] J. Steil, A. Bulling. "Discovery of Everyday Human Activities from Long-Term Visual Behavior Using Topic Models," in *Proc. UBICOMP*, 2015, pp.7-11.
- [3] K.Sharma, P. Jermann, M. Nussli and P. Dillenbourg. "Understanding collaborative program comprehension: Interlacing gaze and dialogues," in *Proc. CSCL*, 2013, pp.15-19.
- [4] <https://pupil-labs.com/pupil/>.
- [5] <https://pupil-labs.com/blog/2013-12/pupil-v0-3-6-marker-tracking>
- [6] <https://github.com/pupil-labs/pupil-docs/blob/master/user-docs/pupil-capture.md>
- [7] <http://www.jdvorak.eu/eye-tracking>
- [8] <https://docs.pupil-labs.com/>
- [9] <https://docs.pupil-labs.com/#pupil-hardware>
- [10] <https://github.com/pupil-labs/pupil/issues/753>
- [11] <https://www.mathworks.com/help/matlab/ref/table.html?requestedDomain=true>
- [12] <http://www.yorku.ca/jdc/Matlab/Lesson1.htm>